PrepPath

Software Design Document

Version	1.0 — Hackathon MVP
Context	Internal hackathon — interview tracking within the organisation
Frontend	React (Vite)
Backend	Python FastAPI
LLM Provider	Groq API — open source models
Database	SQLite (dev) / PostgreSQL (demo)

This document describes PrepPath: an internal tool for tracking and preparing candidates for interviews within the organisation. It covers architecture, user roles, AI agents, channel progression, and the question difficulty system.

1. Purpose

PrepPath is an internal interview preparation and tracking tool. It assigns candidates to structured learning paths, mentors them through an AI chat interface, and tracks their readiness for client or internal interviews. Managers (Executives) can see all candidates and their readiness status in one place.

It is not a job board or recruitment tool. It tracks preparation quality, not hiring decisions.

1.1 Users

Role	What they do
Executive	Views the dashboard. Sees all candidates, their current level, readiness score, and interview-ready status. Read-only.
Feeder	Creates and publishes interview preparation paths. Assigns paths to candidates.
Candidate	Takes tests, chats with the Mentor AI, runs mock interviews, tracks their own progress.

2. Architecture

Five layers. Each has one responsibility. The React frontend talks only to the FastAPI backend. LLM calls happen only in the AI agents layer.

Figure 1 — System layers

Layer	Role
React Frontend	Single-page app. Three views gated by role. Communicates with the backend over HTTP and WebSocket for streaming.
FastAPI Backend	Five routers. Orchestrates all logic. Calls agents and core modules. Reads and writes to the database.
Core Logic	Plain Python. No LLM. Topic gate, question selector, preliminary test builder. Fast enough to run on every chat turn.
AI Agents	Four modules backed by Groq-hosted open source models. Called synchronously (no task queue).
Data	PostgreSQL. In-process dict for session cache. Groq API for LLM.

3. Frontend — React (Vite + TypeScript)

One React app, three views. Role is read from the JWT on login and routes are protected accordingly.

Route	Role	Purpose
/dashboard	Executive	Table of all candidates: name, channel, readiness score, interview-ready badge. Click a candidate to see their history.
/pipeline	Feeder	Create and edit interview paths. Add sections, concepts, evaluation patterns, seed questions. Publish button triggers question generation.
/chat	Candidate	Mentor chat window with streaming. Shows current concept and a topic coverage progress bar.
/test	Candidate	Preliminary test on first visit. Used for all advancement assessments. Shows results and feedback after submit.
/mock	Candidate	Interview simulation. Mentor acts as interviewer. Debrief shown at the end.
/progress	Candidate	Read-only view: current channel, covered concepts, gaps, strengths, readiness score.

4. Backend — FastAPI routers

Key endpoints

Endpoint	What it does
PUT /candidates/{id}/pipeline	Assign a path. Triggers preliminary test build and Plan Gen Agent.
POST /pipelines/{id}/publish	Publish a path. Triggers Question Gen Agent to expand seeds into question bank.
POST /sessions/{id}/chat	Main chat turn. Calls Mentor Agent, parses signal tag, updates topic gate, fires Evaluator if gate triggers.
GET /sessions/{id}/gate-status	Returns topic coverage % — polled by the progress bar in /chat.
POST /assessments/{id}/submit	Submit test answers. Calls Evaluator Agent. Writes channel + level + gaps back to candidate.
GET /candidates/{id}/progress	Full progress data for /progress view and Executive dashboard.

All routes require a JWT. Role guards are enforced per endpoint — a Candidate cannot call pipeline endpoints; a Feeder cannot call session endpoints.

5. AI agents

Four agents. All are Python modules called directly (no queue). Each makes one synchronous HTTP call to the Groq API.

Model choices: llama-3.3-70b-versatile for reasoning-heavy agents (Mentor, Question Gen, Plan Gen). llama-3.1-8b-instant for the Evaluator — it only needs to parse structured answers and return JSON, so a smaller faster model is appropriate.

Agent	Model	Triggered by	What it does
Mentor Agent	llama-3.3-70b-versatile (streaming)	Every chat turn	Reads channel, gaps, strengths, candidate plan, section context. Picks a teaching technique. Appends a hidden <signal> tag with concept_tag and answer quality. Does NOT plan — only executes.
Evaluator Agent	llama-3.1-8b-instant (non-streaming)	On test submit, topic gate fire, mock end	Marks answers against the rubric. Returns JSON: { level, channel, gaps[], strengths[], scores }. The ONLY place channel is assigned.
Question Gen Agent	llama-3.3-70b-versatile (non-streaming)	On path publish	Expands each seed question into 9 variants (3 bands × 3 types). Generates rubrics in the same run. Tags every question with concept_tag, difficulty_band, seniority.
Plan Gen Agent	llama-3.3-70b-versatile (non-streaming)	Once per candidate-path assignment	Reads preliminary test results. Creates a personalised plan: section order, concept emphasis, pattern emphasis, difficulty starting points per concept.

5.1 Groq API configuration

```
# .env
GROQ_API_KEY=gsk_...
GROQ_BASE_URL=https://api.groq.com/openai/v1
```

```
# agents/base.py — shared client
from groq import Groq
client = Groq(api_key=os.environ["GROQ_API_KEY"])
```

Groq exposes an OpenAI-compatible API. Streaming is supported on all models. Switching models requires only changing the model string in the agent file.

6. Candidate journey

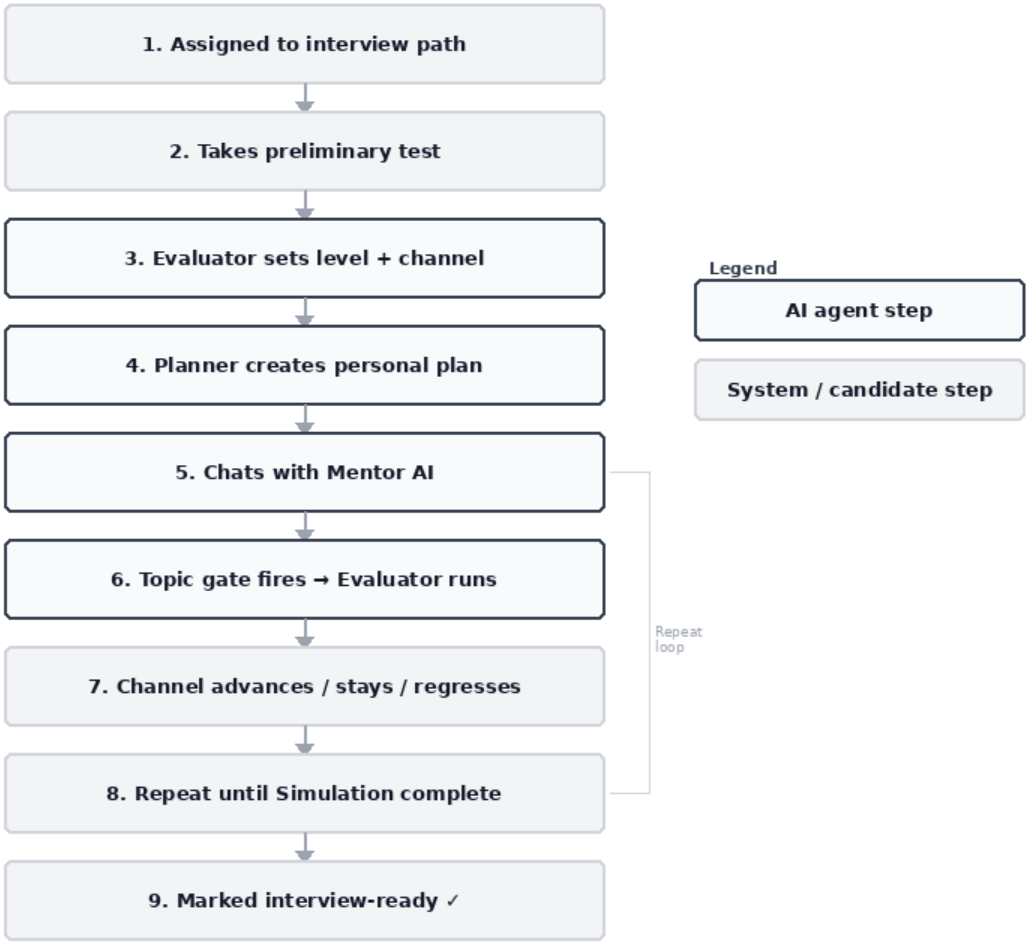

Figure 2 — Candidate journey from assignment to interview-ready

7. Channel progression

Candidates move through three progression channels. Channel assignment is done exclusively by the Evaluator Agent after every assessment.

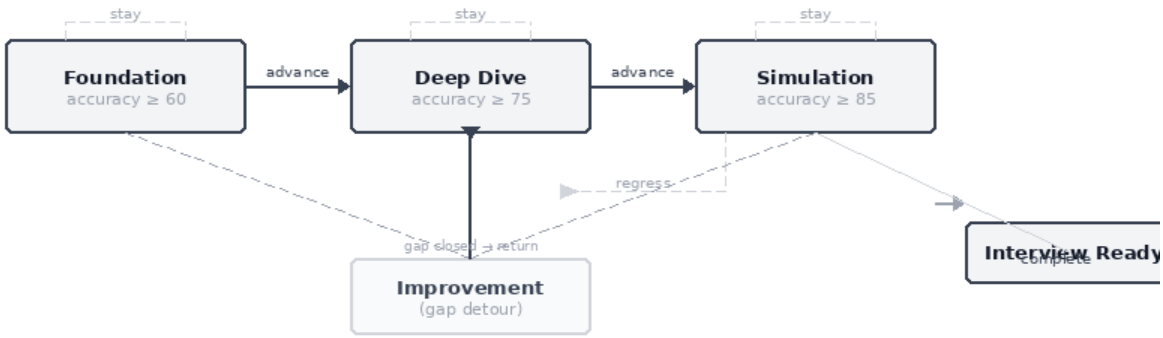

Figure 3 — Channel progression and transitions

Channel	Advancement threshold	Behaviour
Foundation	accuracy ≥ 60, fluency ≥ 50	Mentor teaches before testing. Scaffolding and analogies. No time pressure. Goal: concept coverage at basic correctness.
Deep Dive	accuracy ≥ 75, depth ≥ 65, fluency ≥ 65	Mentor challenges with Socratic probing and constraint injection. Goal: trade-off reasoning and edge-case awareness.
Simulation	accuracy ≥ 85, depth ≥ 80, fluency ≥ 75	Mentor acts as interviewer. No hints. Time pressure. Debrief only at the end. Sets interview_ready = True on completion.
Improvement	Triggered from any channel	Detour — not a progression step. Targets specific gap concepts. Candidate returns to previous channel when gaps close. Tracked via pre_improvement_channel column.

Regression is supported. Deep Dive can drop back to Foundation if accuracy falls below 55. Simulation can drop back to Deep Dive on significant regression.

8. Topic gate and chat turn sequence

The topic gate is the mechanism that decides when to trigger an assessment. It is plain Python — no LLM. It fires when two conditions are both true:

- session.answer_count >= 3 (safety floor — prevents firing on one lucky answer)
- set(covered_concepts) contains all of set(required_concepts)

For the Improvement channel, required_concepts is set to only the gap concepts. When those are covered, the candidate returns to their previous channel.

Figure 4 — What happens on each chat turn

9. Question difficulty

Three bands

Band	Definition
foundational	Recall and basic comprehension. Single concept, unambiguous answer, under 60 seconds.
deepdive	Application and analysis. Multi-step reasoning, at least one trade-off named.
interview_ready	Synthesis and evaluation. Ambiguous framing, time pressure implied, interviewer style.

Band selection at runtime

select_band() in core/question_selector.py decides which band to pull for each turn — no LLM:

- Simulation → always interview_ready
- Improvement → always foundational
- First attempt at a concept → one step below channel ceiling
- Wrong or partial → back to foundational
- Correct on second attempt → ceiling band for this channel

Repetition prevention

- Layer 1 — session exclusion: never repeat a question in the same sitting (in-memory set, zero DB cost).
- Layer 2 — history exclusion: skip questions the candidate already answered correctly (one DB read).
- Fallback: if pool empty, drop history filter. If still empty, Mentor asks freeform.

10. Database

Table	Key columns
users	id, email, name, role, hashed_pw
candidates	id, user_id, path_id, plan_id, channel, level, gaps[], strengths[], pre_improvement_channel, interview_ready, readiness_score
learning_paths	id, name, target_role, seniority, tech_stack[], description, status
sections	id, path_id, name, description, order_index, core_concepts[]
eval_patterns	id, section_id, name, description, sample_patterns[]
questions	id, section_id, pattern_id, concept_tag, difficulty_band, seniority, pattern_type, text, is_preliminary, status
rubrics	id, section_id, pattern_id, seniority, correct_criteria, partial_criteria, disqualifiers
sessions	id, candidate_id, section_id, covered_concepts[], required_concepts[], answer_count, status
session_answers	id, session_id, question_id, candidate_text, quality, concept_tag
assessments	id, candidate_id, trigger, level_assigned, channel_assigned, gaps[], scores{}, feedback_text
candidate_plans	id, candidate_id, section_order[], concept_weights{}, pattern_weights{}, difficulty_start{}
candidate_question_history	id, candidate_id, question_id, last_quality, times_seen

In-process Python dicts (FastAPI app memory): session_id → last 6 chat turns · session_id → set of question IDs used this session. These reset on server restart — acceptable for a hackathon.

11. Build order

When	What to build
Day 1 AM	DB schema, JWT auth, user and candidate CRUD, role guard middleware.
Day 1 PM	Pipeline router, Feeder view (create path, sections, patterns, seeds, publish), Question Gen Agent — verify questions appear in DB.
Day 2 AM	Preliminary test builder, Assessment router, Evaluator Agent, candidate test UI — verify channel and gaps written after submit.
Day 2 PM	Plan Gen Agent, Session router (the orchestrator), Mentor Agent with streaming, topic gate, chat UI with coverage bar.
Day 3 AM	Question selector, question history table, mock interview mode, channel advancement and regression end-to-end.
Day 3 PM	Executive dashboard with interview_ready flag, candidate progress view, demo data, end-to-end test run.

If time runs short: prioritise Mentor Agent (demo centrepiece), then Executive dashboard (business value), then mock interview mode (nice to have).

12. Stack and environment

Component	Technology
Frontend	React 18 + Vite + TypeScript + Tailwind CSS
Backend	Python 3.11 + FastAPI + SQLAlchemy + Uvicorn
LLM	Groq SDK — llama-3.3-70b-versatile + llama-3.1-8b-instant
Database	SQLite (local) / PostgreSQL (demo)
Auth	JWT (python-jose) + bcrypt

```
# Required .env variables
GROQ_API_KEY=gsk_...
DATABASE_URL=sqlite:///./preppath.db
JWT_SECRET=your-secret
VITE_API_URL=http://localhost:8000
```

— end of document —